Assignment 4

CODE

```
#include <iostream>
#include <iomanip>
using namespace std;

#define INF 999

int main()
{
    int n;

    cout << "Enter number of nodes: ";
    cin >> n;

    int cost[20][20], dist[20][20], nextHop[20][20];

    // Input cost matrix
    cout << "\nEnter the cost matrix:\n";
    cout << "Enter 999 for infinity (no direct connection).\n\n";

    for (int i = 0; i < n; i++)
    {
        cout << "Row " << i + 1 << ": ";

        for (int j = 0; j < n; j++)
        {
            cin >> cost[i][j];

            // Initial distance is the same as cost
            dist[i][j] = cost[i][j];

            // Set next hop
            if (i == j)
                nextHop[i][j] = -1;
            else if (cost[i][j] != INF)
                nextHop[i][j] = j;
            else
                nextHop[i][j] = -1;
        }
```

```
}

// Display initial routing tables
cout << "\n========== INITIAL ROUTING TABLES ==========\n";

for (int i = 0; i < n; i++)
{
    cout << "\nRouter " << i + 1 << endl;
    cout << "Destination\tCost\tNext Hop\n";

    for (int j = 0; j < n; j++)
    {
        cout << j + 1 << "\t\t";

        if (dist[i][j] == INF)
            cout << "INF\t";
        else
            cout << dist[i][j] << "\t";

        if (nextHop[i][j] == -1)
            cout << "-\n";
        else
            cout << nextHop[i][j] + 1 << "\n";
    }
}

// Distance Vector Algorithm
bool changed;

do
{
    changed = false;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            for (int k = 0; k < n; k++)
            {
                // Check if i can reach k and k can reach j
                if (cost[i][k] != INF && dist[k][j] != INF)
                {
                    int newCost = cost[i][k] + dist[k][j];
```

```
                if (newCost < dist[i][j])
                {
                    dist[i][j] = newCost;
                    nextHop[i][j] = nextHop[i][k];
                    changed = true;
                }
            }
        }
    }

} while (changed);

// Display final routing tables
cout << "\n========== FINAL ROUTING TABLES ==========\n";

for (int i = 0; i < n; i++)
{
    cout << "\nRouter " << i + 1 << endl;
    cout << "Destination\tCost\tNext Hop\n";

    for (int j = 0; j < n; j++)
    {
        cout << j + 1 << "\t\t";

        if (dist[i][j] == INF)
            cout << "INF\t";
        else
            cout << dist[i][j] << "\t";

        if (nextHop[i][j] == -1)
            cout << "-\n";
        else
            cout << nextHop[i][j] + 1 << "\n";
    }
}

// Find shortest path
int source, destination;

cout << "\nEnter source node: ";
cin >> source;

cout << "Enter destination node: ";
```

```
    cin >> destination;

    source--;
    destination--;

    if (dist[source][destination] == INF)
    {
        cout << "\nNo path exists between the selected nodes.\n";
    }
    else
    {
        cout << "\n========== SHORTEST PATH ==========\n";

        cout << "Path: ";

        int current = source;

        cout << current + 1;

        while (current != destination)
        {
            current = nextHop[current][destination];
            cout << " -> " << current + 1;
        }

        cout << "\nMinimum Cost: "
             << dist[source][destination] << endl;
    }

    return 0;
}
```

OUTPUT

```
Enter number of nodes: 4

Enter the cost matrix:
Enter 999 for infinity (no direct connection).

Row 1: 0 2 5 10
Row 2: 2 0 999 1
Row 3: 5 999 0 2
Row 4: 10 1 2 0

========== INITIAL ROUTING TABLES ==========

Router 1
Destination     Cost    Next Hop
1               0       -
2               2       2
3               5       3
4               10      4

Router 2
Destination     Cost    Next Hop
1               2       1
2               0       -
3               INF     -
4               1       4

Router 3
Destination     Cost    Next Hop
1               5       1
2               INF     -
3               0       -
4               2       4

Router 4
Destination     Cost    Next Hop
1               10      1
2               1       2
3               2       3
4               0       -
```

```
========== FINAL ROUTING TABLES ==========

Router 1
Destination      Cost     Next Hop
1                0        -
2                2        2
3                5        3
4                3        2

Router 2
Destination      Cost     Next Hop
1                2        1
2                0        -
3                3        4
4                1        4

Router 3
Destination      Cost     Next Hop
1                5        1
2                3        4
3                0        -
4                2        4

Router 4
Destination      Cost     Next Hop
1                3        2
2                1        2
3                2        3
4                0        -

Enter source node: 1
Enter destination node: 4

========== SHORTEST PATH ==========
Path: 1 -> 2 -> 4
Minimum Cost: 3


...Program finished with exit code 0
Press ENTER to exit console.
```